

Weekly Learning Debrief

MIS410 — Data Analytics Certificate | Week 4

Topics: Cohort analysis (SQL + Python), pandas Series indexing, nested dictionaries, retention metrics, heatmap visualization

Reading: Cohort Analysis Foundations

Covered cohort analysis — grouping entities (customers, users, patients) by a shared starting point in time, then tracking how each group behaves afterward. Read a companion walkthrough building monthly acquisition cohorts in pandas, plus textbook coverage of the four core cohort metric types.

- **Retention:** did they come back? (repeated engagement over time)
- **Survivorship:** are they still here at all, regardless of activity level?
- **Returnship:** did they act again past a minimum threshold (e.g. a repeat purchase)?
- **Cumulative:** running total by a fixed point — used for customer lifetime value (LTV/CLTV)
- **COALESCE:** SQL's fallback function — returns the first non-null value from a list. The SQL equivalent of pandas' `.fillna()`.
- Reinforced the `first_value` indexing trick from the Chapter 3 sales reading: freeze a starting value, divide every later value by it to get a percentage relative to a fixed baseline — the same logic later used for cohort retention rate.

Pandas Series: Indexing & Nested Dictionaries

Worked through the different ways to pull values out of a labeled pandas Series, and how nested dictionaries map directly onto DataFrames.

	Label-based	Position-based
Single value	<code>obj["b"]</code> / <code>obj.loc["b"]</code>	<code>obj[1]</code> / <code>obj.iloc[1]</code>
Multiple values	<code>obj[["b", "a", "d"]]</code>	<code>obj[[1,3]]</code> / <code>.iloc[[0]]</code>
Meaning	"Find the row named this"	"Find whatever is in this numbered slot"

- Boolean indexing (`obj[obj < 2]`) filters using a True/False condition applied to every row — the same underlying mechanic as `.isnull()` and the cohort filtering used later in this week's assignment.
- Nested dictionary rule: outer keys become DataFrame columns, inner keys become rows — a dictionary is essentially a table that hasn't been drawn yet.

Assignment: SQL + Python Cohort Analysis

Built a full cohort retention pipeline across two related tables (customers, purchases) — 100 customers, 500 purchases, spanning nearly 5 years of sign-up dates. Covered database setup, SQL cohort/retention queries, Python execution and rate calculation, pivot/heatmap visualization, and a written interpretation report.

Tool	What it does
<code>DATE_FORMAT(date, '%Y-%m')</code>	SQL: reformats a full date down to just year-month, used to define a cohort label
<code>TIMESTAMPDIFF(MONTH, d1, d2)</code>	SQL: calculates whole months between two dates — “months since sign-up” for each purchase

<code>JOIN ... ON customer_id</code>	SQL: connects two tables (customers, purchases) using their shared key
<code>COUNT(DISTINCT col)</code>	SQL: counts unique values, avoiding double-counting a customer with multiple purchases
<code>.map(lookup_series)</code>	pandas: looks up a value from another Series and attaches it to every matching row
<code>.pivot_table(index, columns, values)</code>	pandas: reshapes long data into a grid of rows, columns, and cell values
<code>sns.heatmap(data, annot, fmt, cmap)</code>	Seaborn: draws a pivot table as a colored grid with labeled values in each cell
<code>.loc[label]</code> vs <code>.iloc[position]</code>	pandas: explicit label-based vs. position-based lookup, the safer modern habit

- **Key self-caught error:** initial cohort size was calculated from purchase activity at month 0, which produced retention rates over 100% — mathematically impossible. Root cause: not every customer's first purchase happens in their sign-up month. Fixed by pulling true cohort size directly from the customers table (actual sign-up counts).
- **Design decision:** the full heatmap spanned 51+ months since sign-up, mostly sparse given the dataset's size — trimmed to a 12-month window using `cohort_pivot.iloc[:, :12]` for readability.
- **Findings:** retention dropped off noticeably by months 4–5; February and March cohorts retained more consistently than others — but with small sample sizes (often 1–3 customers per cell), these patterns were treated as observations worth further investigation, not confirmed trends.

Discussion: SQL + Python for Cohort Analysis in Practice

Applied this week's assignment directly to a hockey season-ticket-holder scenario: cohort = first ticket-purchase year, retention = renewal rate in subsequent seasons. Correctly identified that SQL creates and structures the cohorts (via `DATE_FORMAT` and `JOIN`), while Python performs the retention math and visualization on top of already-formed cohorts — not the other way around.

Debugging & Reasoning Patterns Worth Remembering

- **A retention/percentage result over 100% is always a red flag**, not just messy data — it signals a flawed assumption in the calculation, worth tracing back before trusting any visualization built on top of it.
- **Observed pattern $\neq$ confirmed trend**, especially with small sample sizes. A drop from 100% to 50% can mean “2 of 2 people bought” became “1 of 2,” not a meaningful behavioral shift — worth naming that distinction explicitly in written interpretation rather than overstating certainty.
- **A chart looking “ugly” can have two separate causes:** a math error upstream, or a genuine scope/readability problem — diagnose which one before assuming the visualization library itself is at fault.
- **SQL and Python have a division of labor:** SQL is well-suited to defining/structuring groups directly in the database (`DATE_FORMAT`, `JOIN`); Python is well-suited to the statistical math and visualization on top of that structure.

Next steps: bring this same format back each week to track growth toward the Mod 8 goal of working independently.